

Projet de développement orienté données (partie 2)

1. Modélisation multidimensionnelle	3
1.1. Magasin de données n°1 : maintenance aéronautique	3
1.1.1. Schéma en étoile	3
1.1.2. Choix des dimensions et des mesures	3
1.1.3. Dictionnaire des données	4
1.1.4. Niveau logique : transformation R-OLAP	8
1.1.5. Niveau physique : création des vues SQL associées	8
1.2. Magasin de données n°2 : marketing et profil des voyageurs	10
1.2.1. Schéma en étoile	10
1.2.2. Choix des dimensions et des mesures	11
1.2.3. Dictionnaire des données	12
1.2.4. Niveau logique : transformation R-OLAP	15
1.2.5. Niveau physique : création des vues SQL associées	15
2. Complétion de l'entrepôt de données	17
2.1. Les limites du jeu de données initial	17
2.2. Solution mise en œuvre : génération automatisée	18
3. Processus ETL sous Power BI	18
3.1. Méthodes d'accès aux sources de données	18
3.2. Transformations réalisées	18
4. Restitutions et rapports	19
4.1. Objectifs d'analyse et indicateurs	19
4.1.1. Pour le groupe "Maintenance Aéronautique"	19
4.1.2. Pour le groupe "Marketing"	19
4.2. Choix et description des visuels de restitution	20
4.2.1. Tableau de bord "Maintenance et Exploitation"	20
4.2.2. Tableau de bord "Analyse Marketing et Sociologique"	21
5. Bilan	22
5.1. Répartition des tâches	22
5.2. Difficultés rencontrées et solutions apportées	23
5.2.1. Génération d'un jeu de données cohérent (Script Python)	23
5.2.2. Conception des vues SQL pour le modèle R-OLAP	23

Lien du répertoire github contenant tous les fichiers du projet :

<https://github.com/Marshall331/projet-entrepot-donnees>

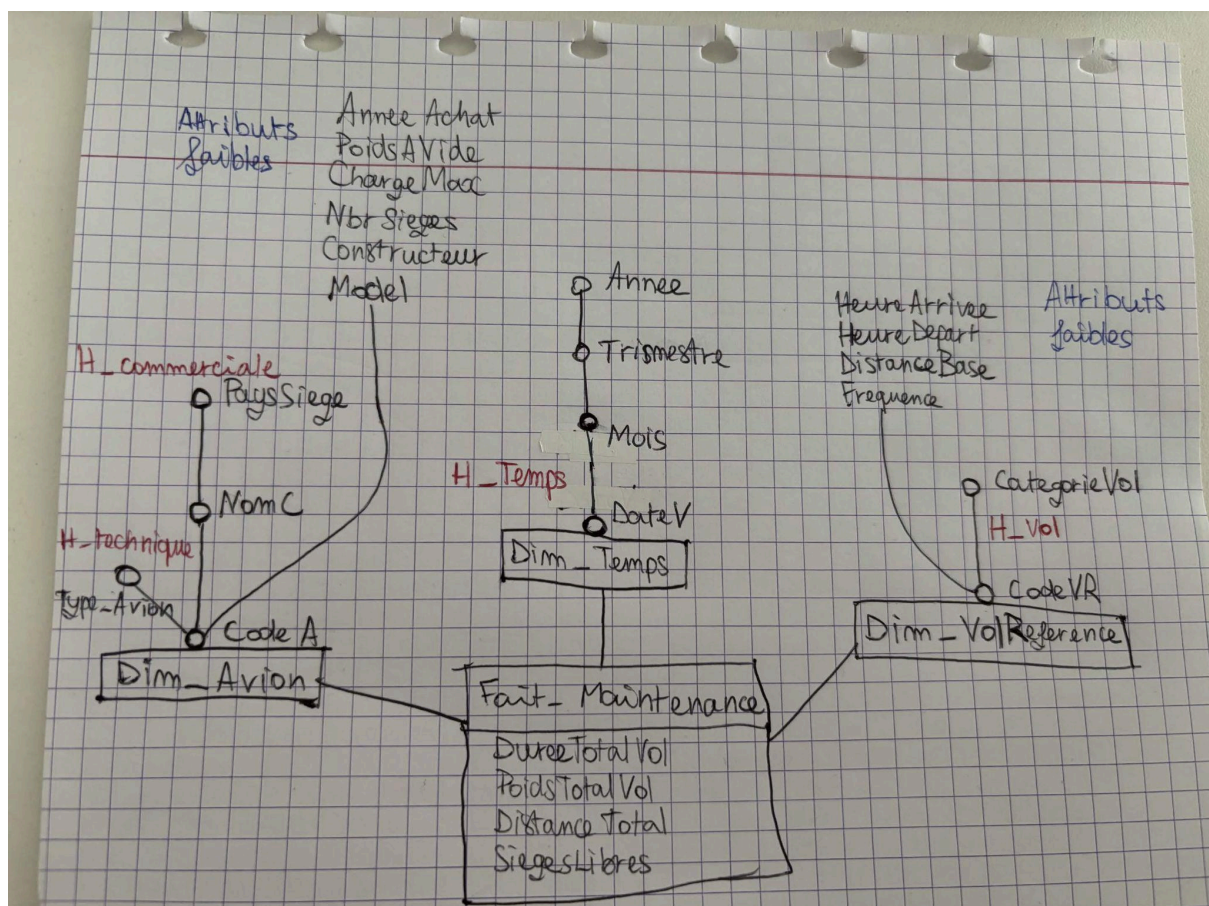
1. Modélisation multidimensionnelle

1.1. Magasin de données n°1 : maintenance aéronautique

Ce premier magasin de données est configuré spécifiquement pour répondre aux besoins des ingénieurs de maintenance aéronautique, qui supervisent l'utilisation et l'usure de la flotte d'avions du consortium.

1.1.1. Schéma en étoile

Le schéma en étoile retenu est centré autour d'une table de faits principale connectée à trois tables de dimensions :



1.1.2. Choix des dimensions et des mesures

La table de faits (**Fait_Maintenance**) :

Elle enregistre chaque événement (chaque vol effectué par un avion à une date donnée).

Elle agrège quatre mesures clés :

DureeTotalVol : somme simple de la durée réelle passée en l'air (mesure d'usure des moteurs).

PoidsTotalVol : représente la charge totale subie par la structure au décollage. Elle est calculée dynamiquement par le calcul :
$$\text{PoidsTotalVol} = \text{PoidsAVide (de l'avion)} + \text{ChargeTotale (fret/passagers)} + \text{ChargeKerosene}$$

DistanceTotal : cumul de la distance parcourue par l'appareil (exprimée en km).

SiegesLibres : indicateur d'optimisation commerciale et de charge, calculé par la différence entre la capacité de l'appareil et l'affluence réelle :
$$\text{SiegesLibres} = \text{NbrSieges (de l'avion)} - (\text{NbrPassagersEco} + \text{NbrPassagersAffaire} + \text{NbrPassagersPremiere})$$

Les tables de Dimensions (axes d'analyse) :

Voici les trois dimensions :

Dim_Avion : regroupe l'ensemble des informations de l'appareil. Le Type d'avion (ex: gros-porteur), le nom de la compagnie propriétaire ainsi que le pays de son siège social y sont directement intégrés afin d'éviter de devoir joindre les tables d'origine Types, Compagnies et Pays.

Dim_VolReference : contient les axes structurels du trajet (fréquence, distance théorique, heures et catégorie de vol).

Dim_Temps : dimension indispensable pour gérer la granularité temporelle demandée (analyse par année, par trimestre et par mois) sans dépendre des fonctions de date natives parfois lourdes.

1.1.3. Dictionnaire des données

Ce dictionnaire formalise les métadonnées de notre schéma en étoile, en précisant les règles d'extraction (ETL) depuis la base opérationnelle (OLTP).

Code	Désignation	Type	Extraction
DureeTotalVol	Durée effective passée en vol	Réel (Mesure)	<code>Vols.DureeVol</code>
PoidsTotalVol	Poids total de l'appareil au décollage	Réel (Mesure)	<code>= Vols.ChargeTotale + Vols.ChargeKerosene</code>
DistanceTotal	Distance parcourue lors du vol	Entier (Mesure)	<code>VolsReferences.Distance</code>
SiegesLibres	Nombre de sièges invendus sur le vol	Entier (Mesure)	<code>= Avions.NbrSieges - (Vols.NbrPassagersEco + Vols.NbrPassagersAffaire + Vols.NbrPassagersPremiere)</code>
CodeA	Numéro de licence identifiant l'avion	Texte	<code>Avions.CodeA</code>
Modele	Modèle de l'avion	Texte	<code>Avions.Modele</code>
Constructeur	Constructeur de l'avion	Texte	<code>Avions.Constructeur</code>

NbrSieges	Capacité maximale de passagers	Entier	Avions.NbrSieges
ChargeMax	Poids total autorisé en charge	Réel	Avions.ChargeMax
PoidsAvide	Poids de l'appareil vide	Réel	Avions.PoidsAVide
AnneeAchat	Année d'acquisition	Entier	Avions.AnneeAchat
TypeAvion	Typologie (Cargo, Gros porteur...)	Texte	Types.NomTy
CodeC	Code AITA de la compagnie	Texte	Compagnies.CodeC
NomC	Nom complet de la compagnie	Texte	Compagnies.NomC
PaysSiege	Pays où se trouve le siège social	Texte	Pays.NomP

CodeVR	Identifiant du vol de référence	Texte	<code>VolsReferences.CodeVR</code>
Frequence	Rythme du vol de référence	Texte	<code>VolsReferences.Frequence</code>
DistanceBase	Distance théorique du trajet	Entier	<code>VolsReferences.Distance</code>
HeureDepart	Heure locale de départ	Heure	<code>VolsReferences.HeureDepart</code>
HeureArrivee	Heure locale d'arrivée	Heure	<code>VolsReferences.HeureArrivee</code>
CategorieVol	Catégorie (Commercial, Officiel...)	Texte	<code>Categories.NomCa</code>
DateV	Date du vol	Date	<code>Vols.DateV</code>
Annee	Année du vol	Entier	<code>= EXTRACT(YEAR FROM Vols.DateV)</code>
Mois	Mois du vol	Entier	<code>= EXTRACT(MONTH FROM Vols.DateV)</code>

Trimestre	Trimestre du vol	Entier	= <code>EXTRACT(QUARTER FROM VoIs.DateV)</code>
------------------	---------------------	--------	--

1.1.4. Niveau logique : transformation R-OLAP

Conformément à l'architecture R-OLAP (Relational OLAP), nous traduisons notre modèle conceptuel multidimensionnel en un schéma relationnel classique, dénormalisé pour l'analyse. Les clés primaires sont soulignées et les clés étrangères vers les dimensions sont marquées d'un #.

DIM_AVION (CodeA, Modele, Constructeur, NbrSieges, ChargeMax, PoidsAVide, AnneeAchat, TypeAvion, CodeC, NomC, PaysSiege)

DIM_VOLREFERENCE (CodeVR, Frequence, DistanceBase, HeureDepart, HeureArrivee, CategorieVol)

DIM_TEMPS (DateV, Annee, Mois, Trimestre)

FAIT_MAINTENANCE (CodeA#, CodeVR#, DateV#, DureeTotalVol, PoidsTotalVol, DistanceTotal, SiegesLibres)

1.1.5. Niveau physique : création des vues SQL associées

-- 1. Vue pour la Dimension Avion

CREATE OR REPLACE VIEW Vue_Dim_Avion AS

SELECT

 a.CodeA,

 a.Modele,

 a.Constructeur,

 a.NbrSieges,

 a.ChargeMax,

 a.PoidsAVide,

 a.AnneeAchat,

 t.NomTy AS TypeAvion,


```
c.CodeC,  
  
c.NomC,  
  
p.NomP AS PaysSiege  
  
FROM Avions a  
  
JOIN Types t ON a.CodeTy = t.CodeTy  
  
JOIN Compagnies c ON a.CodeC = c.CodeC  
  
JOIN Pays p ON c.CodeP = p.CodeP;
```

-- 2. Vue pour la Dimension Vol de Référence

```
CREATE OR REPLACE VIEW Vue_Dim_VolReference AS  
  
SELECT  
  
    CodeVR,  
  
    Frequence,  
  
    Distance AS DistanceBase,  
  
    HeureDepart,  
  
    HeureArrivee,  
  
    cat.NomCa AS CategorieVol  
  
FROM VolsReferences vr  
  
JOIN Categories cat ON vr.CodeCa = cat.CodeCa;
```

-- 3. Vue pour la Dimension Temps (Générée à partir des dates de vols)

```
CREATE OR REPLACE VIEW Vue_Dim_Temps AS  
  
SELECT DISTINCT  
  
    DateV,  
  
    EXTRACT(YEAR FROM DateV) AS Annee,  
  
    EXTRACT(MONTH FROM DateV) AS Mois,  
  
    EXTRACT(QUARTER FROM DateV) AS Trimestre
```

```
FROM Vols;
```

-- 4. Vue pour la Table de Faits Maintenance (Calculs des mesures)

```
CREATE OR REPLACE VIEW Vue_Fact_Maintenance AS
```

```
SELECT
```

```
    v.CodeA,
```

```
    v.CodeVR,
```

```
    v.DateV,
```

```
    v.DureeVol AS DureeTotalVol,
```

```
    (v.ChargeTotale + v.ChargeKerosene) AS PoidsTotalVol,
```

```
    vr.Distance AS DistanceTotal,
```

```
    (a.NbrSieges - (v.NbrPassagersEco + v.NbrPassagersAffaire + v.NbrPassagersPremiere))  
    AS SiegesLibres
```

```
FROM Vols v
```

```
JOIN Avions a ON v.CodeA = a.CodeA
```

```
JOIN VolsReferences vr ON v.CodeVR = vr.CodeVR;
```

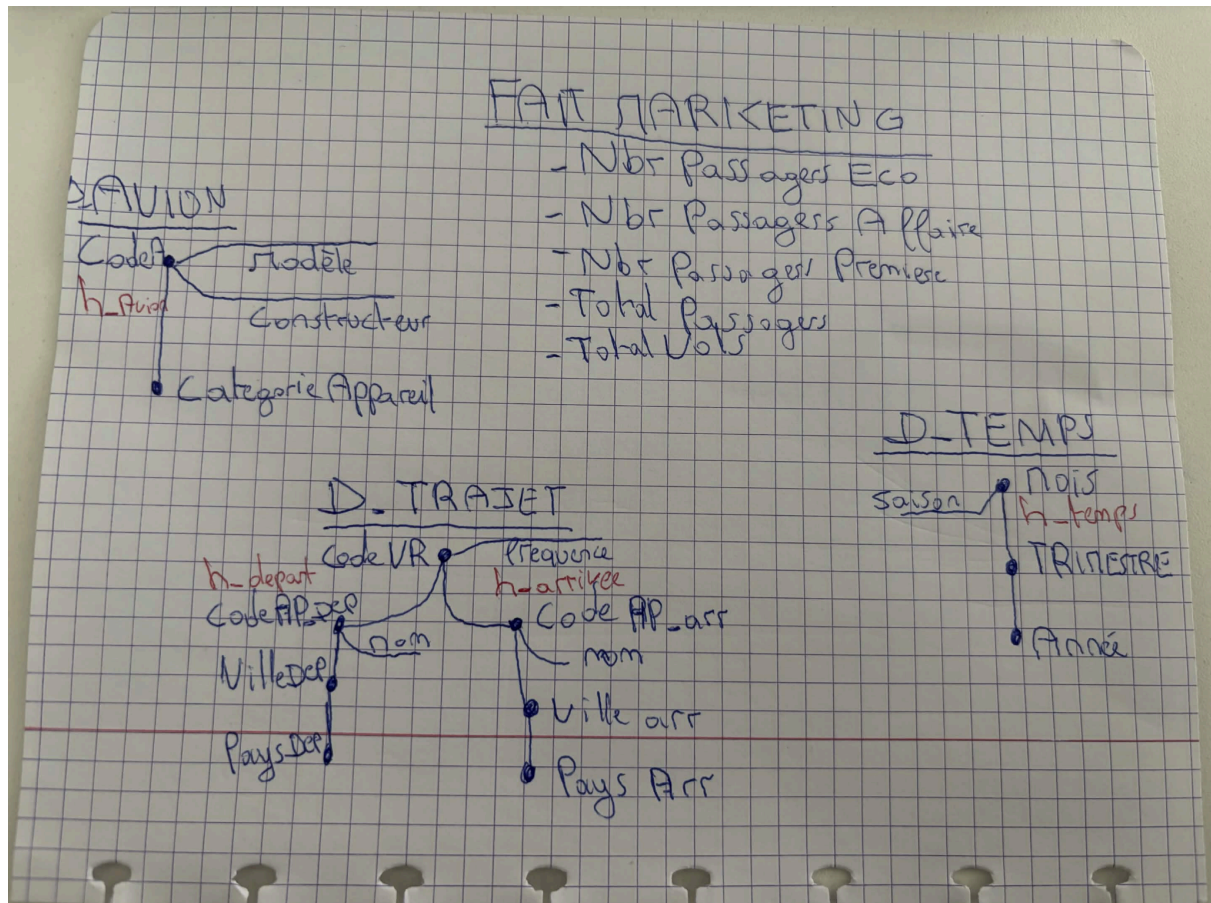
1.2. Magasin de données n°2 : marketing et profil des voyageurs

Ce second magasin de données est conçu spécifiquement pour les directeurs marketing du consortium. Son objectif est de fournir une vision claire des habitudes de consommation, de la saisonnalité des flux et de la typologie socio-économique des passagers afin d'optimiser l'ouverture de nouvelles lignes ou d'ajuster les offres commerciales.

1.2.1. Schéma en étoile

Pour répondre à ces besoins, nous avons mis en place un second schéma en étoile dénormalisé. Il isole les volumes de trafic au centre et déploie les axes géographiques, temporels et techniques tout autour.

Voici le code PlantUML permettant de générer automatiquement le diagramme de ce modèle décisionnel :



1.2.2. Choix des dimensions et des mesures

La table de faits (**Fact_Marketing**) :

Elle capte l'activité commerciale de chaque vol à une date précise. Contrairement au premier magasin de données, elle se concentre uniquement sur des indicateurs de décompte humain :

Elle agrège quatre mesures clés :

NbrPassagersEco, NbrPassagersAffaire, NbrPassagersPremiere : conservés de manière distincte pour servir de marqueurs socio-économiques et analyser la répartition des profils voyageurs.

TotalPassagers : mesure calculée (Éco + Affaire + Première) permettant d'analyser globalement le volume brut de trafic.

TotalVols : un indicateur de comptage simple (COUNT) pour mesurer la densité des rotations d'avions sur un axe donné.

Les tables de Dimensions (axes d'analyse) :

Voici les trois dimensions :

Dim_Trajet : c'est la clé de voûte de l'analyse des déplacements. Elle fusionne le vol de référence et les caractéristiques complètes de l'aéroport de départ (origine) et de l'aéroport d'arrivée (destination). Avec les données comme le nom de l'aéroport, la ville et le pays pour le départ et l'arrivée, le directeur marketing pourra filtrer les flux à l'échelle d'une ville (ex: de Paris) ou d'un pays entier (ex: vers les États-Unis).

Dim_Avion_Marketing : elle se concentre sur les attributs descriptifs de la flotte, en y ajoutant une colonne calculée stratégique : la *CategorieAppareil*. Cet attribut classe dynamiquement l'avion en tant qu'"Avion de ligne" (long/moyen-courrier) ou "Avion régional" (court-courrier) selon le type d'appareil d'origine, répondant précisément à la demande d'analyse sociologique du comportement de voyage.

Dim_Temps_Marketing : indispensable pour suivre l'évolution des comportements dans le temps (comparaisons d'une année sur l'autre, tendances par trimestre). Nous y avons ajouté l'attribut *Saison* (Hiver, Printemps, Été, Automne) calculé en fonction du mois du vol, permettant de mettre en évidence la saisonnalité des flux touristiques ou d'affaires.

1.2.3. Dictionnaire des données

Ce dictionnaire recense l'ensemble des données souhaitées pour ce magasin multidimensionnel, en précisant leur type et leurs règles d'extraction ou de calcul depuis la base OLTP (PostgreSQL).

Code	Désignation	Type	Extraction
NbrPassagersEco	Nombre de passagers en classe économique	Entier (Mesure)	<code>VoIs.NbrPassagersEco</code>
NbrPassagersAffaire	Nombre de passagers en classe affaire	Entier (Mesure)	<code>VoIs.NbrPassagersAffaire</code>

NbrPassagersPremiere	Nombre de passagers en première classe	Entier (Mesure)	<code>Vols.NbrPassagersPremiere</code>
TotalPassagers	Volume total de passagers du vol	Entier (Mesure)	<code>= Vols.NbrPassagersEco + Vols.NbrPassagersAffaire + Vols.NbrPassagersPremiere</code>
TotalVols	Compteur de vols pour évaluer la densité	Entier (Mesure)	<code>= 1</code> (par ligne de vol transactionnelle)
CodeA	Numéro de licence identifiant l'avion	Texte	<code>Avions.CodeA</code>
Modele	Modèle de l'avion	Texte	<code>Avions.Modele</code>
Constructeur	Constructeur de l'avion	Texte	<code>Avions.Constructeur</code>
CategorieAppareil	Typologie de l'avion (Ligne / Régional)	Texte	<code>= IF Avions.NbrSieges <= 150 THEN 'Avion</code>

			Régional' ELSE 'Avion de Ligne'
CodeVR	Identifiant du vol de référence	Texte	<code>VolsReferences.CodeVR</code>
Frequence	Rythme du vol de référence	Texte	<code>VolsReferences.Frequence</code>
CodeAP_Dep	Code de l'aéroport de départ	Texte	<code>VolsReferences.CodeAP_Dep</code>
NomAP_Dep	Nom complet de l'aéroport de départ	Texte	<code>Airports.NomAP</code> (Aéroport Départ)
CodeAP_Arr	Code de l'aéroport d'arrivée	Texte	<code>VolsReferences.CodeAP_Arr</code>
NomAP_Arr	Nom complet de l'aéroport d'arrivée	Texte	<code>Airports.NomAP</code> (Aéroport Arrivée)
DateV	Date à laquelle s'opère le vol	Date	<code>Vols.DateV</code>

Annee	Année du vol	Entier	= EXTRACT(YEAR FROM Vols.DateV)
Mois	Mois du vol	Entier	= EXTRACT(MONTH FROM Vols.DateV)
Trimestre	Trimestre du vol	Entier	= EXTRACT(QUARTER FROM Vols.DateV)
Saison	Saison météorologique du vol	Texte	= CASE MONTH(Vols.DateV) IN (12,1,2) THEN 'Hiver' ...

1.2.4. Niveau logique : transformation R-OLAP

De la même manière que pour la maintenance, nous traduisons notre modèle conceptuel multidimensionnel en un schéma relationnel (architecture R-OLAP). Les clés primaires sont soulignées et les clés étrangères vers les dimensions sont marquées d'un #.

DIM_AVION_MARKETING (CodeA, Modele, Constructeur, CategorieAppareil)

DIM_TRAJET (CodeVR, Frequence, CodeAP_Dep, NomAP_Dep, CodeAP_Arr, NomAP_Arr)

DIM_TEMPS_MARKETING (DateV, Annee, Mois, Trimestre, Saison)

FAIT_MARKETING (CodeA#, CodeVR#, DateV#, NbrPassagersEco, NbrPassagersAffaire, NbrPassagersPremiere, TotalPassagers, TotalVols)

1.2.5. Niveau physique : création des vues SQL associées

-- 1. Vue pour la Dimension Avion Marketing (Création de la Catégorie)

CREATE OR REPLACE VIEW Vue_Dim_Avion_Marketing AS

SELECT

```

CodeA,

Modele,

Constructeur,

CASE

    WHEN NbrSieges <= 150 THEN 'Avion Régional'

    ELSE 'Avion de Ligne'

END AS CategorieAppareil

FROM Avions;

```

-- 2. Vue pour la Dimension Trajet (Simplifiée selon le DDL disponible)

```

CREATE OR REPLACE VIEW Vue_Dim_Trajet AS

SELECT

    vr.CodeVR,

    vr.Frequence,

    -- Branche Départ

    ad.CodeAP AS CodeAP_Dep,

    ad.NomAP AS NomAP_Dep,

    -- Branche Arrivée

    aa.CodeAP AS CodeAP_Arr,

    aa.NomAP AS NomAP_Arr

FROM VolsReferences vr

JOIN Airports ad ON vr.CodeAP_Dep = ad.CodeAP

JOIN Airports aa ON vr.CodeAP_Arr = aa.CodeAP;

```

-- 3. Vue pour la Dimension Temps Marketing (Avec Saisonnalité)

```

CREATE OR REPLACE VIEW Vue_Dim_Temps_Marketing AS

SELECT DISTINCT

```



```

DateV,

EXTRACT(YEAR FROM DateV) AS Annee,

EXTRACT(MONTH FROM DateV) AS Mois,

EXTRACT(QUARTER FROM DateV) AS Trimestre,

CASE

    WHEN EXTRACT(MONTH FROM DateV) IN (12, 1, 2) THEN 'Hiver'

    WHEN EXTRACT(MONTH FROM DateV) IN (3, 4, 5) THEN 'Printemps'

    WHEN EXTRACT(MONTH FROM DateV) IN (6, 7, 8) THEN 'Eté'

    WHEN EXTRACT(MONTH FROM DateV) IN (9, 10, 11) THEN 'Automne'

END AS Saison

FROM Vols;

```

-- 4. Vue pour la Table de Faits Marketing

```

CREATE OR REPLACE VIEW Vue_Fact_Marketing AS

SELECT

    CodeA,

    CodeVR,

    DateV,

    NbrPassagersEco,

    NbrPassagersAffaire,

    NbrPassagersPremiere,

    (NbrPassagersEco + NbrPassagersAffaire + NbrPassagersPremiere) AS TotalPassagers,

    1 AS TotalVols

FROM Vols;

```

2. Complétion de l'entrepôt de données

2.1. Les limites du jeu de données initial

Lors de la Phase 1, nous avons inséré manuellement un volume restreint de données transactionnelles (environ 30 vols).

Cependant, la Phase 2 nécessite un volume de données significatif et varié pour exploiter un schéma en étoile et alimenter des tableaux de bord interactifs (tendances, saisonnalité). Sans cela, les graphiques seraient plats et les filtres croisés non pertinents.

2.2. Solution mise en œuvre : génération automatisée

Nous avons développé un script Python pour générer les données :

Initialisation (SQL) : connexion à PostgreSQL et exécution du script_creation.sql (incluant la création de nos vues SQL).

Création des dimensions : importation des données statiques (Aéroports, Compagnies, Avions).

Génération de données : simulation de l'activité sur deux ans .

Pour chaque avion et chaque jour, le script génère des données réalistes :

- remplissage aléatoire (50-100%).
- répartition socio-économique (80% Éco, 15% Affaires, 5% Première).
- calcul physique des charges (Kérosène et fret) en respectant la ChargeMax de l'avion.

Chargement massif : Insertion des vols générés en mémoire via un buffer StringIO dans la table Vols.

3. Processus ETL sous Power BI

3.1. Méthodes d'accès aux sources de données

Au lieu d'importer les tables brutes, nous avons connecté Power BI directement aux vues SQL que nous avons créées dans PostgreSQL (Vue_Fact_Maintenance, Vue_Dim_Avion, etc).

3.2. Transformations réalisées

Grâce à la création de nos vues SQL en amont, les transformations dans Power Query ont été réduites à leur strict minimum.

Pour les mesures : tout a été pré-calculé par nos vues SQL (le calcul des SiegesLibres, du poids total, ou le regroupement des pays et villes).

Création des dimensions Temps : la seule transformation significative maintenue dans Power Query fut la génération de la Dim_Temps (et Dim_Temps_Marketing). À partir de la colonne DateV, nous avons utilisé l'éditeur pour extraire les Années, Trimestres, Mois, et pour ajouter une colonne conditionnelle de Saison (Hiver, Été, etc.) indispensable pour l'analyse Marketing.

4. Restitutions et rapports

Une fois les modèles de données importés et liés dans Power BI, nous avons conçu deux tableaux de bord interactifs distincts pour répondre aux problématiques spécifiques de nos deux groupes d'utilisateurs.

4.1. Objectifs d'analyse et indicateurs

4.1.1. Pour le groupe "Maintenance Aéronautique"

L'objectif principal de ce groupe est de surveiller l'usure de la flotte et d'optimiser l'exploitation technique des appareils. Les indicateurs clés (KPIs) suivis sont :

- **L'usure matérielle** : via la somme des durées de vol (**DureeTotalVol**) et l'accumulation des distances parcourues (**DistanceTotal**).
- **Les contraintes structurelles** : via l'analyse du poids total supporté par les appareils au décollage, incluant le kérosène (**PoidsTotalVol**).
- **L'optimisation des capacités** : via le suivi du nombre de sièges inoccupés par vol (**SiegesLibres**).

4.1.2. Pour le groupe "Marketing"

L'objectif de la direction marketing est d'extraire des tendances sociologiques, de comprendre les flux de déplacement et d'analyser le profil des voyageurs. Les indicateurs clés (KPIs) suivis sont :

- **Le volume de trafic** : via le nombre total de passagers et le nombre de rotations (**TotalVols**).

- **Le profil socio-économique** : analysé à travers la répartition exacte des voyageurs par classe de cabine (`NbrPassagersEco`, `NbrPassagersAffaire`, `NbrPassagersPremiere`).
- **Les habitudes de voyage** : mises en évidence par l'analyse croisée des volumes selon la saisonnalité, la géographie (origines/destinations) et le type de ligne (Avion régional vs Avion de ligne).

4.2. Choix et description des visuels de restitution

Pour rendre ces indicateurs exploitables, nous avons sélectionné des visuels adaptés à la nature des données.

4.2.1. Tableau de bord "Maintenance et Exploitation"

Pour ce rapport, nous avons mis en place un bandeau de filtres (Segments) permettant aux ingénieurs de cibler leur analyse par :

- année/Mois
- compagnie
- modèle d'avion

Les visuels choisis sont :

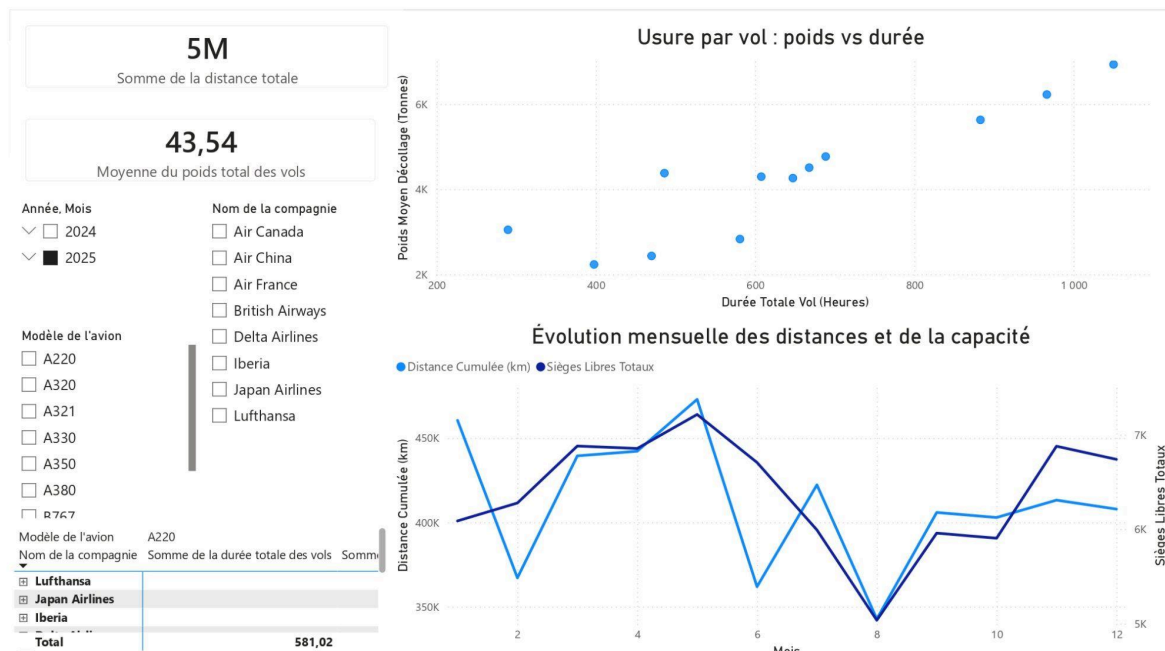
Graphique en courbes (Évolution mensuelle) : ce visuel répond à la demande d'analyse temporelle. Il place les mois en axe X et trace deux courbes : la `DistanceTotal` et le nombre de `SiegesLibres`. En filtrant sur un avion précis, l'ingénieur peut visualiser instantanément les pics d'utilisation ou les anomalies de remplissage.

Graphique en nuage de points (analyse de charge critique) : pour identifier les vols les plus usants pour la flotte. Nous avons placé la `DureeTotalVol` en axe X et le `PoidsTotalVol` en axe Y, avec chaque vol (`CodeVR`) représenté par un point. Ce visuel permet aux ingénieurs de repérer immédiatement les cas extrêmes (vols très longs avec une charge maximale) nécessitant une maintenance préventive renforcée.

Matrice (Tableau croisé dynamique) : utilisée pour analyser l'usure selon les références de vol. En lignes, nous avons placé la hiérarchie Compagnie > Modèle d'avion > Code de Référence (`CodeVR`). En valeurs, nous affichons la `DureeTotalVol` et le `PoidsTotalVol`. Ce format tabulaire strict permet aux ingénieurs d'exporter facilement les chiffres vers Excel si besoin.

Cartes à plusieurs lignes (KPI) : mises en évidence en haut du rapport pour donner les macro-indicateurs instantanés (Cumul des distances, Poids moyen supporté) pour la sélection en cours.

Aperçu du tableau de bord final :



4.2.2. Tableau de bord "Analyse Marketing et Sociologique"

Ce rapport s'articule autour de segments de filtres orientés client :

- saison (Hiver, Été...)
- catégorie d'appareil (Régional/Ligne)
- Année

Les visuels choisis sont :

Carte géographique (Flow Map) : pour répondre au besoin d'analyse des origines et destinations. Les pays de départ et d'arrivée sont placés sur la carte, et la taille des bulles représente le volume de TotalPassagers. Cela permet d'identifier visuellement les hubs les plus attractifs.

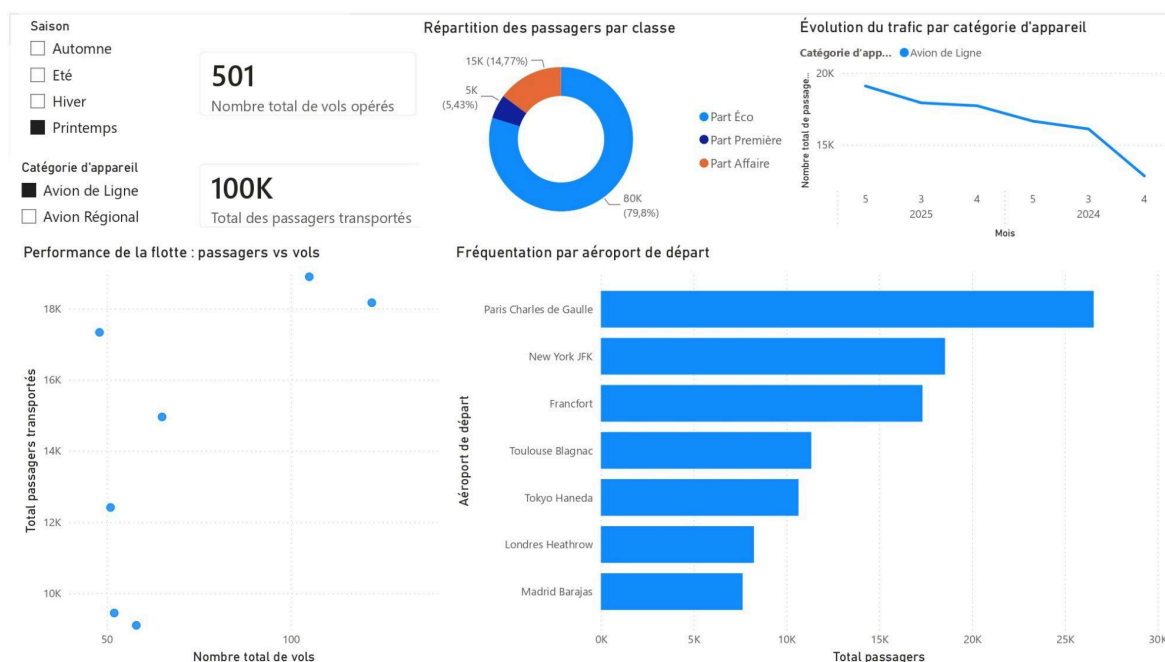
Graphique en barres empilées 100% : idéal pour analyser l'évolution du profil socio-économique dans le temps. L'axe X représente les Trimestres/Mois. Chaque barre représente 100% des passagers, divisée en trois couleurs proportionnelles aux classes (Éco, Affaire, Première). On peut ainsi voir si

la part des voyageurs "Affaire" augmente en hiver ou sur les avions de ligne.

Graphique à secteurs (Donut) : il répartit le volume global des voyageurs selon la CatégorieAppareil (Avion de ligne vs Régional), permettant d'un seul coup d'œil de comprendre les préférences de transport du marché.

Histogramme groupé : utilisé pour l'analyse de la saisonnalité. Il croise les saisons en axe X avec le volume de vols et de passagers en axe Y, confirmant visuellement les périodes de haute et basse affluence.

Aperçu du tableau de bord final :



5. Bilan

5.1. Répartition des tâches

Nous avons opté pour une approche de travail totalement collaborative. Plutôt que de diviser les tâches, nous avons conçu et développé chaque étape ensemble.

5.2. Difficultés rencontrées et solutions apportées

5.2.1. Génération d'un jeu de données cohérent (Script Python)

La difficulté : créer un grand volume de données réalistes tout en respectant l'intégrité référentielle (clés étrangères) et la cohérence métier (calculs de poids, capacités maximales, dates logiques).

La solution : une structuration séquentielle du script (génération des tables "parents" puis "enfants") associée à des règles de gestion strictes dans les boucles de génération. Cela nous a permis d'obtenir un jeu de données robuste et réaliste.

5.2.2. Conception des vues SQL pour le modèle R-OLAP

La difficulté : transformer une base transactionnelle normalisée en un modèle multidimensionnel (schéma en étoile) directement via des vues, ce qui implique des jointures et des colonnes calculées dynamiquement.

La solution : une conception itérative isolant d'abord les dimensions (via EXTRACT et CASE WHEN) puis agrégeant la table de faits centrale. Cette architecture Push-Down (délégation des calculs à PostgreSQL) a rendu l'importation vers Power BI extrêmement rapide et performante.